

Unit-3

Software Quality Assurance & Software Maintenance

1. Quality Concept in Software Engineering

Quality in software engineering refers to the **degree to which software meets specified requirements and satisfies user needs.**

High-quality software should:

- Work correctly
- Be reliable
- Be easy to maintain
- Be secure
- Provide good performance

Software quality ensures that the system is **free from defects and performs efficiently under different conditions.**

Quality in software is usually measured using attributes such as:

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability
- Maintainability
- Flexibility
- Portability

Maintaining software quality is essential because poor quality software can lead to **system failures, financial losses, and user dissatisfaction.**

2. Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a systematic process used to **ensure that software products and development processes meet defined quality standards.**

SQA focuses on **preventing defects rather than detecting them after development.**

The main objectives of SQA are:

- Ensure compliance with software standards
- Improve development processes

- Detect defects early
- Maintain high software quality

SQA activities are performed throughout the **software development lifecycle**.

Activities of SQA

Important SQA activities include:

- Establishing quality standards
- Performing software reviews
- Conducting testing and verification
- Monitoring development processes
- Managing defects
- Ensuring documentation quality

SQA helps organizations produce **reliable and maintainable software systems**.

3. Software Reviews

A **software review** is a process in which software artifacts such as **requirements, design documents, and code are examined by a group of developers**.

The goal of software reviews is to:

- Detect defects early
- Improve software quality
- Ensure compliance with standards
- Improve communication among team members

Reviews can be **formal or informal**.

4. Formal Technical Review (FTR)

A **Formal Technical Review (FTR)** is a structured and systematic review process conducted by a team of technical experts.

The purpose of FTR is to **identify errors in software products before they move to the next development stage**.

Objectives of FTR

- Detect errors in function, logic, or implementation
- Verify that software meets requirements
- Ensure that standards are followed
- Improve software quality

Process of Formal Technical Review

The typical steps in an FTR include:

1. **Planning** – Identify the review team and schedule the review meeting.
2. **Preparation** – Reviewers study the software documents before the meeting.
3. **Review Meeting** – Team members discuss defects and improvement suggestions.
4. **Rework** – The developer corrects identified defects.
5. **Follow-up** – The review leader ensures that corrections are implemented.

Formal technical reviews help reduce development costs because **errors are detected early in the development process**.

5. Software Reliability

Software reliability refers to the **probability that software will operate without failure for a specified period of time under specified conditions**.

Reliability is a critical quality attribute because software failures can lead to serious consequences.

Characteristics of Software Reliability

- Failure-free operation
- Consistent performance
- Error tolerance
- Dependability

Importance of Software Reliability

Reliable software systems:

- Reduce system failures
- Improve user trust
- Lower maintenance costs
- Increase system stability

Reliability can be improved by:

- Proper design
 - Thorough testing
 - Fault detection and correction
 - Error handling mechanisms
-

6. Software Quality Standards

Software quality standards provide **guidelines and best practices to maintain software quality**.

Important quality standards include:

- ISO 9000
- Capability Maturity Model (CMM)

- Six Sigma
-

7. ISO 9000 Standard

ISO 9000 is a family of international standards developed by the **International Organization for Standardization (ISO)**.

It focuses on **quality management systems and process improvement**.

Organizations that follow ISO 9000 standards ensure that:

- Development processes are standardized
- Quality management practices are followed
- Customer satisfaction is maintained

Benefits of ISO 9000

- Improved product quality
 - Better documentation and processes
 - Increased customer confidence
 - International recognition
-

8. Capability Maturity Model (CMM)

The **Capability Maturity Model (CMM)** is a framework developed by the **Software Engineering Institute (SEI)** to improve software development processes.

CMM defines **five maturity levels**.

CMM Levels

1. **Initial Level**
Processes are unpredictable and poorly controlled.
2. **Repeatable Level**
Basic project management practices are established.
3. **Defined Level**
Standardized development processes are documented.
4. **Managed Level**
Processes are measured and controlled.
5. **Optimizing Level**
Continuous improvement of processes.

CMM helps organizations improve **software quality and productivity**.

9. Six Sigma for Software Engineering

Six Sigma is a quality improvement methodology that focuses on **reducing defects and improving process performance**.

The main objective of Six Sigma is to achieve **very low defect rates**.

Six Sigma follows a structured methodology known as **DMAIC**.

DMAIC Process

1. **Define** – Identify the problem and project goals.
2. **Measure** – Measure current performance.
3. **Analyze** – Identify root causes of defects.
4. **Improve** – Implement solutions to remove defects.
5. **Control** – Monitor processes to maintain improvements.

Six Sigma helps improve **software quality, efficiency, and reliability**.

10. SQA Plan

An **SQA Plan (Software Quality Assurance Plan)** is a document that describes **how quality assurance activities will be performed in a software project**.

It ensures that quality standards and procedures are followed.

Components of an SQA Plan

- Quality objectives
- Quality standards
- Roles and responsibilities
- Review and audit procedures
- Testing strategies
- Documentation standards
- Risk management

The SQA plan acts as a **roadmap for ensuring software quality during development**.

11. Software Maintenance

Software maintenance refers to the process of **modifying and updating software after it has been delivered to users**.

Maintenance is required because software must adapt to:

- Changing user requirements
- New technologies
- Bug fixes
- Performance improvements

Software maintenance typically consumes **more than 60% of total software development cost**.

12. Types of Software Maintenance

Software maintenance is classified into four types.

1. Corrective Maintenance

Corrective maintenance involves **fixing errors or bugs found in software after deployment**.

Example:

Fixing a login error in a web application.

2. Adaptive Maintenance

Adaptive maintenance involves **modifying software to adapt to new environments**.

Example:

Updating software to work with a new operating system.

3. Perfective Maintenance

Perfective maintenance focuses on **improving performance or adding new features**.

Example:

Adding new functionality to a mobile application.

4. Preventive Maintenance

Preventive maintenance aims to **prevent future problems by improving software structure and reliability**.

Example:

Refactoring code to improve maintainability.

13. Re-Engineering

Software Re-engineering is the process of **analyzing and improving an existing software system to make it more maintainable and efficient**.

It includes activities such as:

- Code restructuring
- Documentation improvement

- System redesign

Re-engineering helps extend the **life of legacy systems**.

14. Reverse Engineering

Reverse Engineering is the process of **analyzing existing software to understand its design and structure without changing its functionality**.

It is commonly used when documentation is missing.

Example:

Studying compiled software to understand its architecture.

15. Forward Engineering

Forward Engineering is the process of **developing a software system from high-level design to implementation**.

It involves:

- Requirement analysis
- System design
- Coding
- Testing

Forward engineering is essentially the **normal process of software development**.

Conclusion

Software Quality Assurance and Software Maintenance play a crucial role in ensuring that software systems remain **reliable, efficient, and maintainable throughout their lifecycle**. SQA focuses on maintaining quality during development, while maintenance ensures that the software continues to meet evolving user needs. Quality standards such as ISO 9000, CMM, and Six Sigma help organizations improve development processes and deliver high-quality software products.